

Static Timing Analysis

Sci.D.Professor Vazgen Melikyan

Synopsys Armenia Educational Department

CONFIDENTIAL INFORMATION

The information contained in this presentation is the confidential and proprietary information of Synopsys. You are not permitted to disseminate or use any of the information provided to you in this presentation outside of Synopsys without prior written authorization.

IMPORTANT NOTICE

In the event information in this presentation reflects Synopsys' future plans, such plans are as of the date of this presentation and are subject to change. Synopsys is not obligated to update this presentation or develop the products with the features and functionality discussed in this presentation. Additionally, Synopsys' services and products may only be offered and purchased pursuant to an authorized quote and purchase order or a mutually agreed upon written contract with Synopsys.

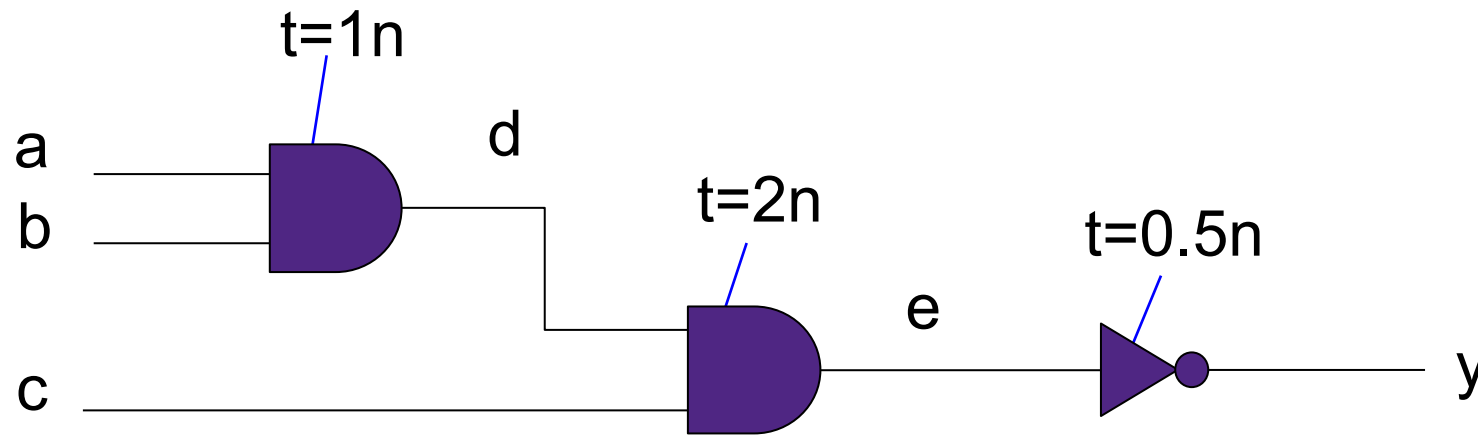
Course Overview

1. STA Concepts
 - 1 lecture
2. Delay Modeling
 - 1 lecture
3. Interconnect Parasitics
 - 1 lecture
4. Delay Calculation
 - 1 lecture
5. Configuring the STA Environment
 - 1 lecture
6. Timing Checks
 - 1 lecture
7. Crosstalk and Noise
 - 1 lecture

2. Delay Modeling

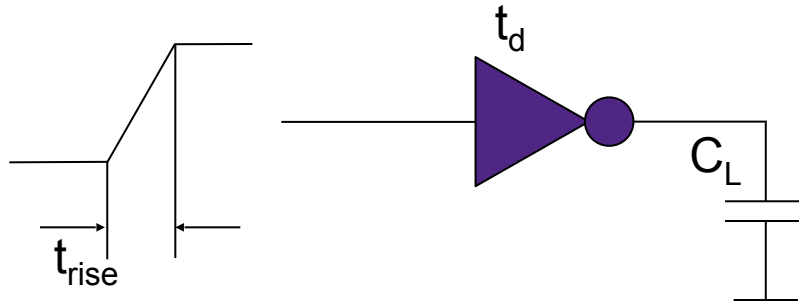


Path Delay: Basic Approach



$$t_{\text{delay}} = 3.5n$$

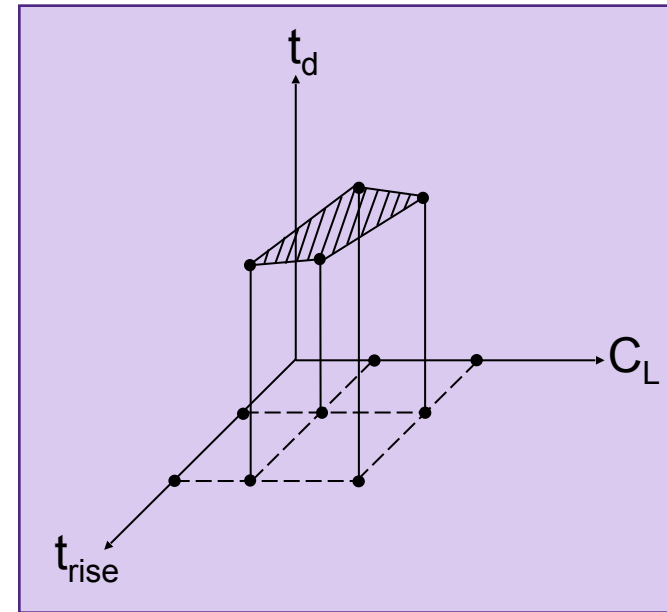
Delay Dependencies



$$t_d \sim C_L$$

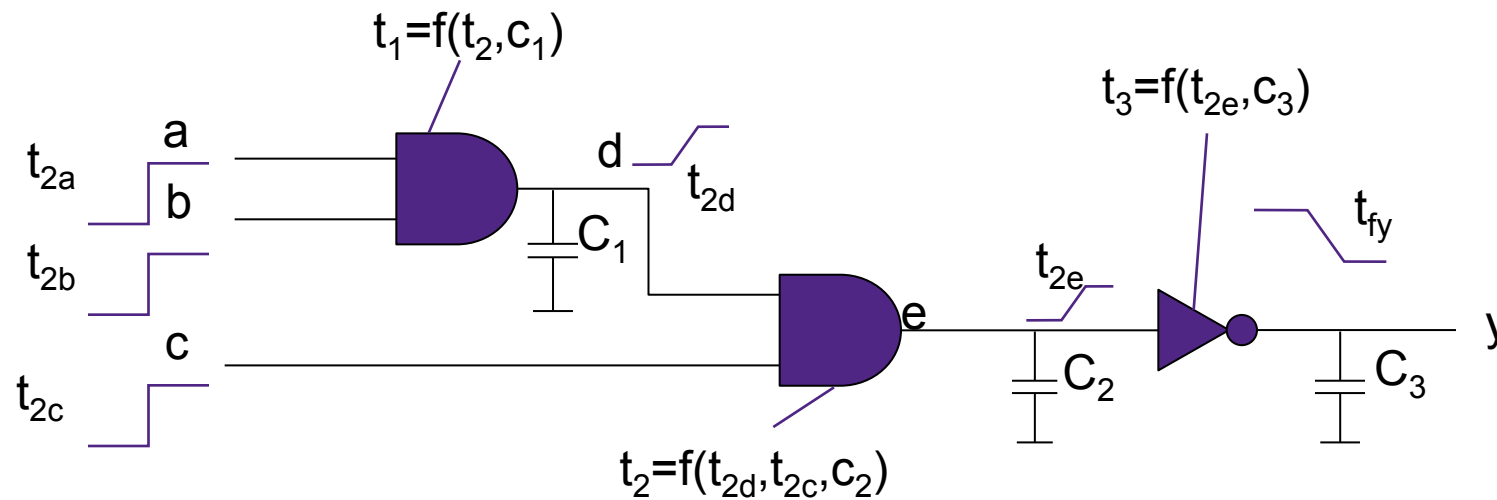
$$t_d \sim t_{\text{rise}}$$

t_{rise}	10ps – 120ps
C_{load}	10fF – 50 fF



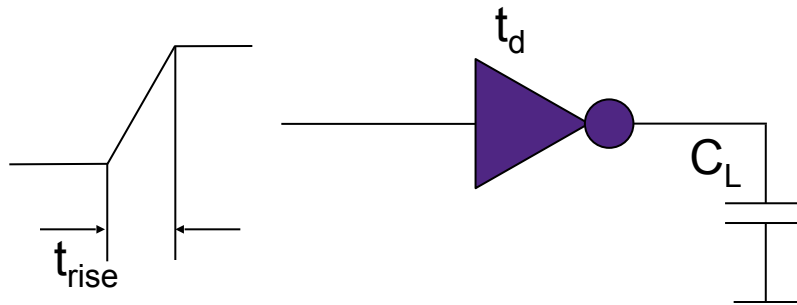
Path Delay Calculation

Specific PVT



$$t_{\text{total}} = f(t_{2a}, t_{2b}, t_{2c}, C_1, C_2, C_3)$$

Operating Conditions



$t_d \sim$ Process Variations

$t_d \sim$ Voltage

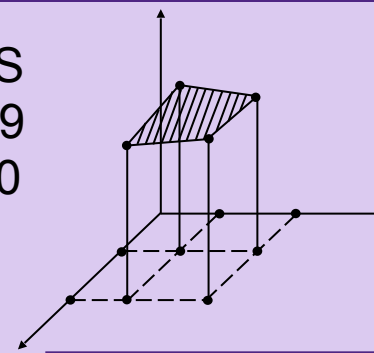
$t_d \sim$ Temperature

Process: TT, FF, SS, etc.

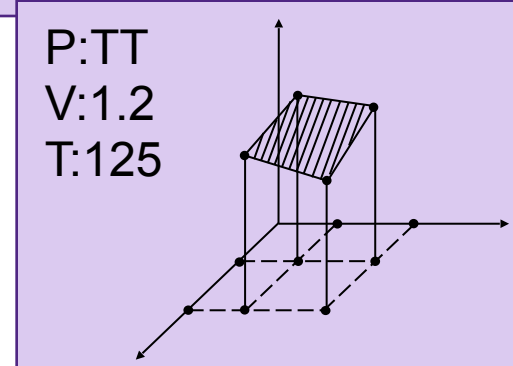
Voltage: $\pm 10\%$

Temperature: $-40 - 125^\circ\text{C}$

P:SS
V:0.9
T:-40



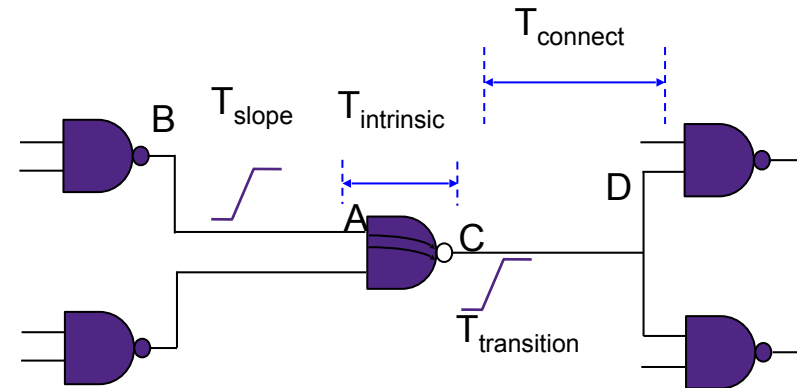
P:TT
V:1.2
T:125



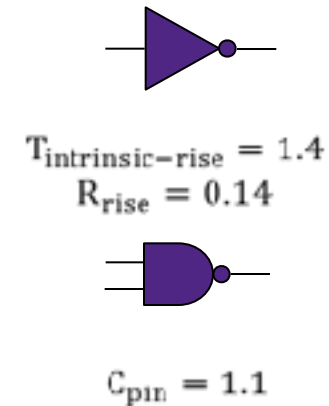
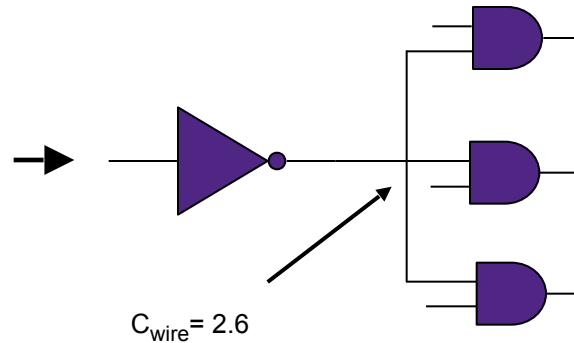
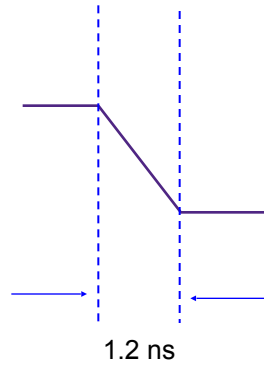
Linear Delay Model

$$T_{total} = T_{slope} + T_{intrinsic} + T_{transition} + T_{connect}$$

- Slope Delay (T_{slope})
 - The transition time of the previous gate
- Intrinsic Delay ($T_{intrinsic}$)
 - Delay of an element
- Transition time ($T_{transition}$)
 - Delay introduced by capacitive load on driving pin
 - $$T_{transition} = R_{drive} * \left(\sum_{pins} C_{pin} + C_{wire} \right)$$
- Connect Delay ($T_{connect}$)
 - Delay from transition of the driving pin to endpoint changing after the



Delay Calculation (Linear) Example



$$\begin{aligned} T_{\text{intrinsic}} &= 1.4 \\ S_{\text{rise}} &= 0.02 \\ R_{\text{rise}} &= 0.14 \\ C_{\text{pins}} &= 3 * (1.1) = 3.3 \\ C_{\text{wire}} &= 2.6 \\ T_{\text{transition}} &= 1.2 \\ T_{\text{connect}} &= 0.7 \end{aligned}$$

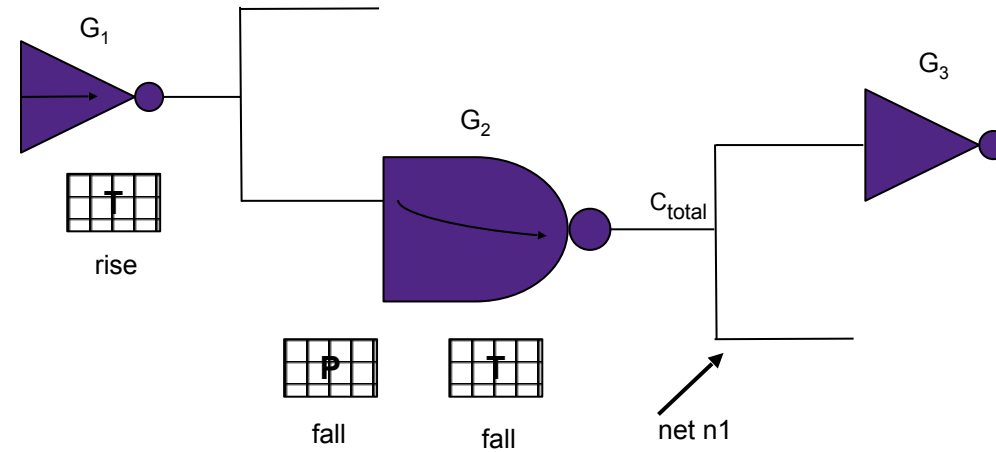
$$\begin{aligned} T_{\text{rise}} &= T_{\text{intrinsic}} + T_{\text{slope}} + T_{\text{transition}} + T_{\text{connect}} \\ &= 1.4 + (1.2 * 0.02) + (0.14)(3.3 + 2.6) + 0.0 = 2.25 \end{aligned}$$

Nonlinear Delay Model

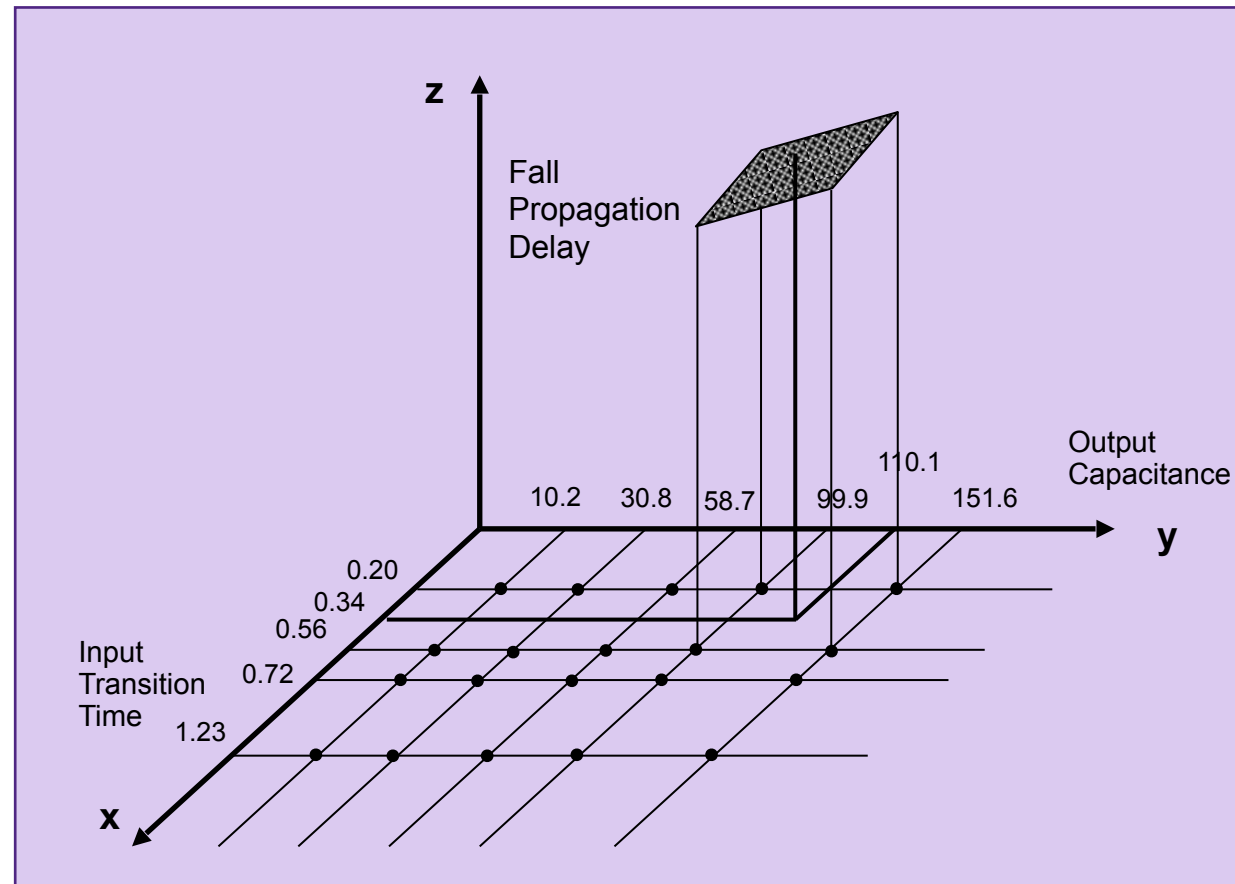
$$T_{\text{total}} = T_{\text{propagation}} + T_{\text{transition}} + T_{\text{connect}}$$

- **Transition time** ($T_{\text{transition}}$)
 - Delay introduced by capacitive load on driving pin (measured, not calculated)
- **Propagation delay** ($T_{\text{propagation}}$)
 - Time from the 50 percent input pin voltage until the gate output just begins to switch (10 percent output voltage) (measured, not calculated)
- **Connect Delay** (T_{connect})
 - Delay from transition of the driving pin (estimated interconnect delay)
- **Transition time and Propagation delay** for each cell are measured beforehand and stored in form of lookup table

Nonlinear Delay Calculation Example



Two-dimensional Interpolation



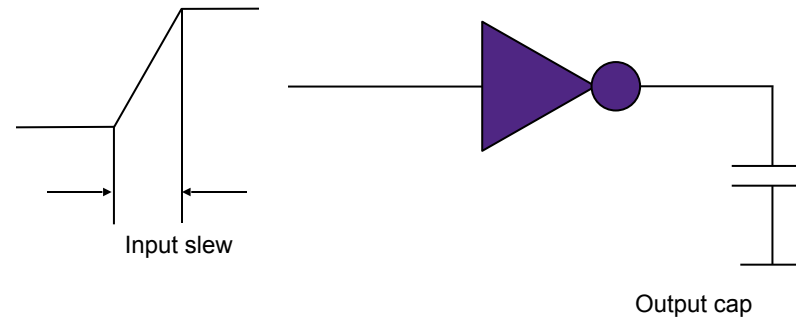
Non Linear Delay Model (NLDM)

cell_rise or cell_fall
table

Input slew

0.7	0.25	0.27	0.31	0.33	0.4
0.5	0.26	0.28	0.29	0.31	0.35
0.2	0.35	0.37	0.39	0.41	0.45
0.1	0.40	0.41	0.43	0.48	0.51

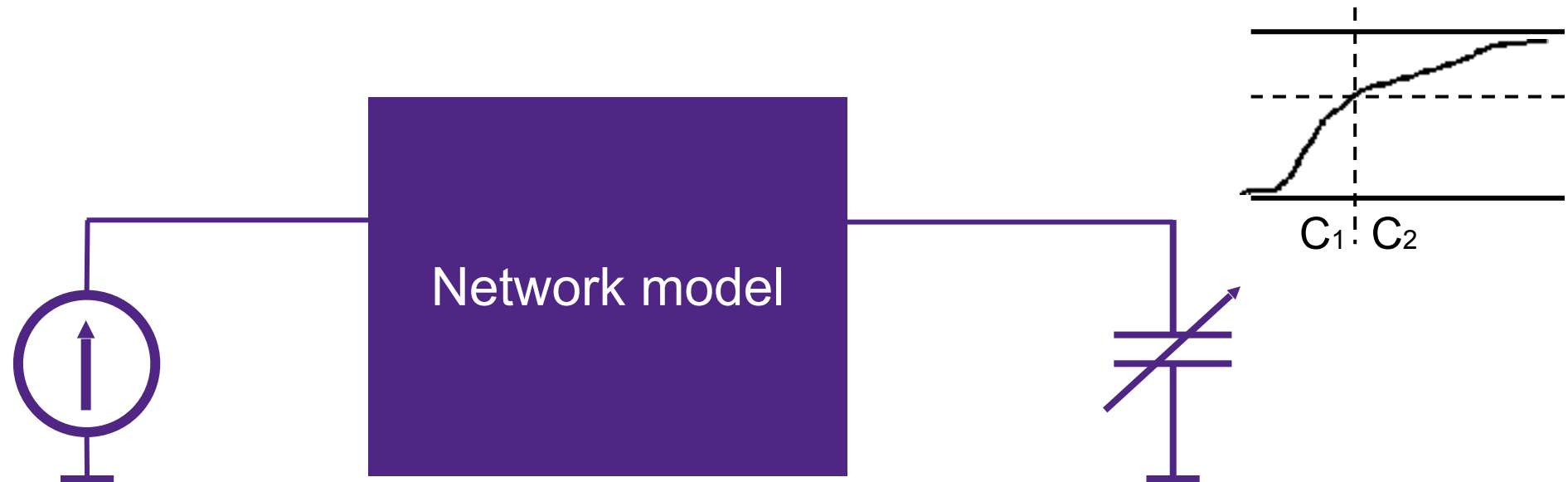
output cap



Delay/Power are measured as function of input slew and output cap

Linear Delay model:
 $\text{Delay} = \text{Intrinsic Delay} + \text{Slope_factor} * \text{Load}(\text{Cap})$

Composite Current Source Model (CCS)



- The **driver model** uses a time-varying current source.
- The **receiver model** consists of 2 different capacitors.
The first one is used as load up to the input delay threshold. A second capacitance value is used when the input waveform reaches the threshold value.
- CCS models are frequently used in advanced technology nodes.

Synopsys Liberty Format (.lib)

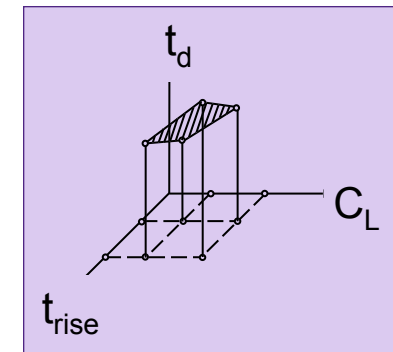
```

library (Digital_Std_Lib) {
  technology (cmos);
  delay_model : table_lookup;
  cell(AND2) {
    area : 2;
    pin(A) {
      direction : input;
    }
    pin(B) {
      direction : input;
    }
    pin(Z) {
      direction : output;
      function : "A*B";
      timing() {
        related_pin : "A" ;
        timing_type : "combinational" ;
        cell_rise(...) {
          index 1("0.016, 0.032, 0.064");
          index 2("2, 4");
          values("1.0020, 1.1280, 3.547 ", \
                "1.0080, 1.1310, 3.847 " );
        }
      }
    } /* end of pin */
  } /* end of cell */
} /* end of library*/

```

Lookup table

		t_{rise}		
		0.016	0.032	0.064
C_L	2	1.0020	1.1280	3.547
	4	1.0080	1.1310	1.1310



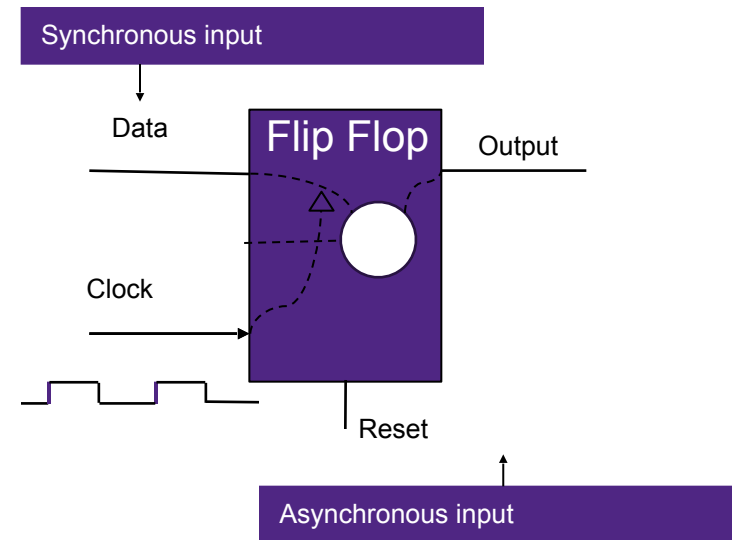
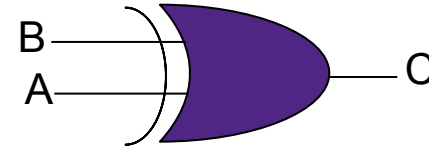
Cell Timing Data

```
library(){  
  lu_table_template ("del_1_7_7") {  
    variable_1 : "input_net_transition";  
    index_1("1, 2, 3, 4, 5, 6, 7");  
    variable_2 : "total_output_net_capacitance";  
    index_2("1, 2, 3, 4, 5, 6, 7");  
  }  
}
```

```
cell (INVX1) {  
  pin (Y) {  
    timing () {  
      related_pin : "A";  
      timing_type : "combinational";  
      timing_sense : "negative_unate";  
      cell_rise ("del_1_7_7") {  
        index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");  
        index_2("0.1, 0.25, 0.5, 1, 2, 4, 8");  
        values("0.016861, 0.0179019, 0.0195185, 0.0229259, 0.029658, 0.043145, 0.07712", \  
              "0.0239648, 0.0255491, 0.0279298, 0.0319930, 0.0387540, 0.0520896, 0.0790211", \  
              "0.0342118, 0.0366966, 0.0402223, 0.0462823, 0.0558327, 0.0705154, 0.0967339", \  
              "0.0491695, 0.0524727, 0.0576512, 0.0665647, 0.0810999, 0.1027237, 0.1342571", \  
              "0.0721332, 0.0765389, 0.0836775, 0.0960890, 0.1171612, 0.1497265, 0.1957640", \  
              "0.1111560, 0.1164417, 0.1252609, 0.1422002, 0.1712097, 0.2171862, 0.2847010", \  
              "0.1841131, 0.1901881, 0.2010298, 0.2194395, 0.2555983, 0.3182710, 0.4139452");  
      }  
    }  
  }  
}
```

Cell Types: Sequential Cells

- Combinational
 - Cell output changes with changes at input
- Sequential
 - Cell has memory and output depends on memory and input



Timing Group Names

N	Parameter	Unit	Symbol	Figure	Definition
1.	Rise transition time rise_transition	ns	t_R		The time it takes a driving pin to make a transition from kV_{DD} to $(1-k)V_{DD}$ value. Usually $k=0.1$ (also possible $k=0.2, 0.3$, etc)
2.	Fall transition time fall_transition	ns	t_F		The time it takes a driving pin to make a transition from $(1-k)V_{DD}$ to kV_{DD} value. Usually $k=0.1$ (also possible $k=0.2, 0.3$, etc)
3.	Propagation delay low-to-high (rise) cell_rise	ns	t_{PLH} (t_{PR})		Time difference between the input signal crossing a $0.5V_{DD}$ and the output signal crossing its $0.5V_{DD}$ when the output signal is changing from low to high
4.	Propagation delay high-to-low (Fall) cell_fall	ns	t_{PHL} (t_{PF})		Time difference between the input signal crossing a $0.5V_{DD}$ and the output signal crossing its $0.5V_{DD}$ when the output signal is changing from high to low

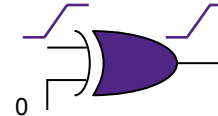
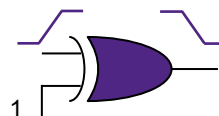
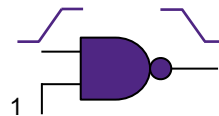
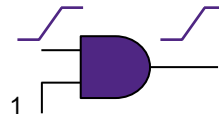
Combinational Timing Arc Syntax

- Combinational timing arc between input A and output Y, with negative dependence, i.e. When A is rising Y is falling and vice-versa

```
cell (INVX1) {
  pin (Y) {
    timing () {
      related_pin : "A";
      timing_type : "combinational";
      timing_sense : "negative_unate";
      cell_rise ("del_1_7_7") {
        index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");
        index_2("0.1, 0.25, 0.5, 1, 2, 4, 8");
        values("0.0168610, 0.0179019, 0.0195185, 0.0229259, 0.0296588, 0.0431451,
0.0702328", \
          "0.0239648, 0.0255491, 0.0279298, 0.0319930, 0.0387540, 0.0520896, 0.0790211", \
          "0.0342118, 0.0366966, 0.0402223, 0.0462823, 0.0558327, 0.0705154, 0.0967339", \
          "0.0491695, 0.0524727, 0.0576512, 0.0665647, 0.0810999, 0.1027237, 0.1342571", \
          "0.0721332, 0.0765389, 0.0836775, 0.0960890, 0.1171612, 0.1497265, 0.1957640", \
          "0.1111560, 0.1164417, 0.1252609, 0.1422002, 0.1712097, 0.2171862, 0.2847010", \
          "0.1841131, 0.1901881, 0.2010298, 0.2194395, 0.2555983, 0.3182710, 0.4139452");
      }
    }
  }
}
```

Delay Analysis

- Calculation of each timing arc's value cell delay or a net delay
 - **Positive unate** timing arc combines rise delays with rise delays and fall delays with fall delays
 - **Negative unate** timing arc combines incoming rise delays with local fall delays and vice versa
 - **Non-unate** timing arc combines local delay with the worst – case incoming delays logic functions whose output value change cannot be predicted



Timing Constraints: Timing Types

Setup/Hold, Recovery/Removal Constraints

N	Parameter	Unit	Symbol	Figure	Definition
1	Setup time (only for flip-flops or latches) setup_rising setup_falling	ns	t_{SU}		The minimum period in which the input data to a flip-flop or a latch must be stable before the active edge of the clock occurs
2..	Hold time (only for flip-flops or latches) hold_rising hold_falling	ns	t_H		The minimum period in which the input data to a flip-flop or a latch must remain stable after the active edge of the clock has occurred
3	Removal time (only for asynchronous Set or Reset) removal_rising , removal_falling	ns	t_{REM}		The minimum time in which the asynchronous Set or Reset pin to a flip-flop or latch must remain enabled after the active edge of the clock has occurred
4	Recovery time (only for asynchronous Set or Reset) recovery_rising , recovery_falling	ns	t_{REC}		The minimum time in which Set or Reset must be held stable after being deasserted before next active edge of the clock occurs

Hold Timing Example

```
pin(D) {
  timing () {
    timing_type : "hold_rising";
    related_pin : "CLK";
    rise_constraint ("vio_7_7_1") {
      index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");
      index_2("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, 1.024");
      values("-0.011, -0.011, -0.014, -0.013, -0.003, 0.025, 0.097", \
        "-0.011, -0.011, -0.015, -0.013, -0.003, 0.028, 0.097", \
        ...
        "-0.015, -0.018, -0.033, -0.063, -0.100, -0.128, -0.228");
    }
    fall_constraint ("vio_7_7_1") {
      ...
    }
  }
}
```

```
lu_table_template ("vio_7_7_1") {
  variable_1 : "related_pin_transition";
  index_1("1, 2, 3, 4, 5, 6, 7");
  variable_2 : "constrained_pin_transition";
  index_2("1, 2, 3, 4, 5, 6, 7");
}
```

Thank You